

Sistemas Operativos

Práctica 3: Sincronización entre procesos

Ejercicio 1

Se tienen dos procesos A y B que ejecutan concurrentemente. No se tiene información sobre cómo serán ejecutados por el *scheduler*. Para cada par A-B detallado a continuación, responder:

- ¿Hay una única salida en pantalla posible para cada proceso?
- Indicar todas las salidas posibles para cada caso.

```
// variables compartidas  
x = 0;
```

```
A():  
  x = x + 1;  
  printf("%d", x)
```

```
B():  
  x = x + 1;
```

```
// variables compartidas  
x = 0;  
y = 0;
```

```
A():  
  for (; x < 4; x++) {  
    y = 0;  
    printf("%d", x);  
    y = 1;  
  }
```

```
B():  
  while (x < 4) {  
    if (y == 1)  
      printf("a");  
  }
```

@ EN LA IZQ NO, A) PUEDE IMPRIMIR 2 o 1, DEPENDE DEL SCHEDULER.

EN LA DERECHA TAMPOCO, B PODRÍA IMPRIMIR A MAS DE 4 VECES.

(b) ALGUN DIA...

Ejercicio 2 ★

Se tiene un sistema con 4 procesos accediendo a una variable compartida x y un *mutex*. Los 4 procesos ejecutan el siguiente código. Ciertas decisiones que toma cada proceso dependen del valor de la variable compartida. Se debe asegurar que cada vez que un proceso lee la variable compartida, previamente solicita el *mutex* y luego lo libera.

¿Estos procesos cumplan con lo planteado? ¿Pueden ser víctimas de *race condition*?

```
x = 0; // Variable compartida
mutex(1); // Mutex compartido

while (1) {
    mutex.wait();
    y = x; // Lectura de x
    mutex.signal();
    if (y <= 5) {
        x++;
    } else {
        x--;
    }
}
```

SeRAN VÍCTIMAS DE RACE CONDITION YA QUE LA PARTE RESALTADA ES CÓDIGO CRÍTICO, ENTONCES EL SIGUIENTE PROCESO AL PRIMERO QUE HITÓ EL `mutex.signal()`, PUEDE LEER O $y = 1$ O $y = 0$. Y ASÍ SUCESIVAMENTE.

Ejercicio 3

La operación `wait()` sobre semáforos suele utilizar una cola para almacenar los pedidos que se encuentran en espera. Si en lugar de una cola utilizara una pila (LIFO), determinar si habrá inanición o funcionará correctamente.

HABRÁ INANICIÓN YA QUE POTENCIALMENTE SIEMPRE PODRÍA HABER UN PROCESO QUE HAGA `wait()` Y POR LO TANTO EL PRIMERO EN LA PILA (NO EL TOPE) HARÍA INDEFINIDAMENTE HASTA VOLVER A EJECUTAR

Ejercicio 4 ★

Demostrar que, en caso de que las operaciones de semáforos `wait()` y `signal()` no se ejecuten atómicamente, entonces se viola la propiedad de exclusión mutua, es decir que un recurso no puede estar asignado a más de un proceso.

Pista: Revisar el funcionamiento interno del `wait()` y del `signal()` mostrados en clase, el cual no se hará de forma atómica, y luego pensar en una traza que muestre lo propuesto.

```
wait() {  
    while (S ≤ 0) { dormir(); }  
    S --;  
}  
  
signal() {  
    S ++;  
    if (ALGUNO ESPERA S) then despertar ALGUNO;  
}
```

ESTO ES PSEUDOCÓDIGO. ES FÁCIL VER QUE NO SON ATÓMICAS PERO SE EJECUTAN DE ESTA MANERA.

ENTONCES, SI NO SE EJECUTASEN DE MANERA ATÓMICA, ES POSIBLE QUE EL SIGUIENTE PSEUDOCÓDIGO NO TENGA EL COMPORTAMIENTO ESPERADO:

```
int X = 0 ;
SEMAFORO sem = sem (1) ;
```

```
int main() {
    while (1)
        sem.Wait()
        x++;
        PRINT(x)
        sem.SIGNAL();
}
```

UN P1 y UN P2 QUE EJECUTEN ESTE PROGRAM
TENDRIAN UNA TRAZA ASI

<u>P1</u>	<u>P2</u>	<u>S</u>
Make - wait		1
VP S = 1		1
INT SCHEDULER	Make - wait	1
	VP S = 1	1
S --	INT SCHEDULER	0
ENTRA A CRIT		0
	S --	-1
	ENTRA A CRIT	-1

y esto sucede porque wait dejó de ser ejecutada de manera atómica y fue interrumpida por el scheduler.

Ejercicio 5

Se tienen n procesos: $P_1, P_2, \dots, P_n$ que ejecutan el siguiente código. Se espera que todos los procesos terminen de ejecutar la función `preparado()` antes de que alguno de ellos llame a la función `critica()`. ¿Por qué la siguiente solución permite inanición? Modificar el código para arreglarlo.

```
preparado()

mutex.wait()
count = count + 1
mutex.signal()

if (count == n)
    barrera.signal()

barrera.wait()

critica()
```

el if no es Atómico y PODRÍA suceder que MAS DE UN PROCESO HAYA COUNT == n + QUE DE PRUE Y SE LLAME A SIGNAL MAS DE UNA VEZ PERO AUN ASI EL RIESGO DE INANICION PARA EL RESTO DE PROCESOS ES VALENTE YA QUE QUE VAYAN HACIENDO WAIT PARA SIEMPRE.

FIX

#DEFINE FORM(i, m) FOR (int i=0; i<m; i++)

SEMAFORO mutex = SEM(1);

SEMAFORO BARRERA = SEM(0);

PREPARADO() {

mutex.wait();

CRITICALLY [COUNT++
if (COUNT == n) FORM(i, m) BARRERA.SIGNAL();
mutex.signal();

BARRERA.wait();

CRITICA();

Ejercicio 6 ★

Cambiar la solución del ejercicio anterior por una solución basada solamente en las herramientas atómicas vistas en las clases, que se implementan a nivel de hardware, y responder las siguientes preguntas:

- ¿Cuál de las dos soluciones genera un código más legible?
- ¿Cuál de ellas es más eficiente? ¿Por qué?
- ¿Qué soporte requiere cada una de ellas del SO y del HW?

Fix Atomico

```
Atomic <int> Count = 0
Atomic <Bool> BARRERA = FALSE

VOID MAIN () {
    PREPARADO();
    COUNT.INC();
    BARRERA.Set ( COUNT == m );
    While ( !BARRERA ) { // busy waiting
        CRITICO();
    }
}
```

(a) Mi fix Atomico es MAS LESIBLE ya que es intuitivo y no requiere entender SEMAFOROS.

(b) EL fix de semaforo es MAS eficiente ya NO hace busy waiting.

(c) SEMAFORO de DIJKSTRA: soporte de SO ya que LA EJECUCION DE wait / signal NO puede ser interrumpida

Atomic: soporte de hardware ya que son INSTRUCCIONES ESPECIALES.

Ejercicio 7 ★

Se tienen N procesos, $P_0, P_1, \dots, P_{N-1}$ (donde N es un parámetro). Se requiere sincronizarlos de manera que la secuencia de ejecución sea $P_i, P_{i+1}, \dots, P_{N-1}, P_0, \dots, P_{i-1}$ (donde i es otro parámetro). Escribir el código que deben ejecutar cada uno de los procesos para cumplir con la sincronización requerida utilizando semáforos (no olvidar los valores iniciales).

```
SEMAFORO MUTEX = SEM(1);  
SEMAFORO arrSEM[N];  
FORM(i, N) arrSEM[i] := SEM(0);  
INT CANTPROCESOS := 0;
```

```
VOID FUNC(N, i) {  
    MUTEX.WAIT();  
    INT INDEX = CANTPROCESOS + 1;  
    MUTEX.SIGNAL();
```

```
    IF (INDEX - 1 == i) {
```

```
        LLAMAR A GOKU();
```

```
        arrSEM[INDEX].SIGNAL(); // ooy de A  $P_{i+1}$ 
```

```
    } ELSE {
```

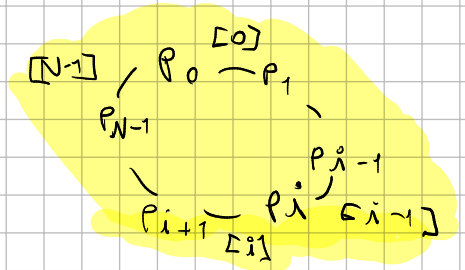
```
        arrSEM[(INDEX - 1 + N) % N].WAIT(); // espero a  $P_{i-1}$ 
```

```
        LLAMAR A GOKU();
```

```
        arrSEM[INDEX].SIGNAL(); // ooy de A  $P_{i+1}$ 
```

```
    }
```

```
}
```



Ejercicio 8 ★

Considerar cada uno de los siguientes enunciados. Para cada caso, escribir el código que permita la ejecución de los procesos según la forma de sincronización planteada utilizando semáforos (no olvidar los valores iniciales). Se debe argumentar porqué cada solución evita la inanición:

1. Se tienen tres procesos (A, B y C). Se desea que el orden en que se ejecutan sea el orden alfabético, es decir que las secuencias normales deben ser: ABC, ABC, ABC, ...

```
SEMAFORO semA = sem(1)
```

```
SEMAFORO semB = sem(0)
```

```
SEMAFORO semC = sem(0)
```

```
VOID A() {
```

```
    While (1) {
```

```
        semA.wait();
```

```
        EJECUTAR PRISIONERO();
```

```
        semB.signal();
```

```
    }
```

```
}
```

```
VOID B() {
```

```
    While (1) {
```

```
        semB.wait();
```

```
        EJECUTAR PRISIONERO();
```

```
        semC.signal();
```

```
    }
```

```
}
```

```
VOID C() {
```

```
    While (1) {
```

```
        semC.wait();
```

```
        EJECUTAR PRISIONERO();
```

```
        semA.signal();
```

```
    }
```

```
}
```

2. Idem anterior, pero se desea que la secuencia normal sea: BBKA, BBKA, BBKA, ...

SEMAFORO semA = sem(0)

SEMAFORO semB = sem(2)

SEMAFORO semC = sem(0)

VOID B() {

INT COUNT = 0;

While (1) {

MACERALGO();

COUNT++;

if (COUNT % 2 == 0) { semC.SIGNAL(); }

semB.Wait();

}

}

VOID C() {

While (1) {

semC.Wait();

MACERALGO();

semA.SIGNAL();

}

}

VOID A() {

While (1) {

semA.Wait();

MACERALGO();

FORM(i, 2) { semB.SIGNAL(); }

}

}

3. Se tienen un productor (A) y dos consumidores (B y C) que actúan no determinísticamente. La información provista por el productor debe ser retirada siempre 2 veces, es decir que las secuencias normales son: ABB, ABC, ACB o ACC. **Nota:** ¡Ojo con la exclusión mutua!

```
SEMAFORO semA = Sem(1);
```

```
SEMAFORO semGen = Sem(0);
```

```
SEMAFORO mutex = Sem(1);
```

```
INT consumidas = 0;
```

```
VOID A() {
```

```
    While (1) {
```

```
        semA.wait();
```

```
        form(i,2) PRODUCIR();
```

```
        semGen.SIGNAL();
```

```
        semGen.SIGNAL();
```

```
    }
```

```
}
```

```
VOID B y C() {
```

```
    While (1) {
```

```
        semGen.wait();
```

```
        mutex.wait();
```

```
        CONSUMIR();
```

```
        CONSUMIDAS++;
```

```
        if (CONSUMIDAS == 2) {
```

```
            semA.SIGNAL();
```

```
            CONSUMIDAS = 0;
```

```
        }
```

```
        mutex.SIGNAL();
```

```
    }
```

```
}
```

4. Se tienen un productor (A) y dos consumidores (B y C). Cuando C retira la información, la retira dos veces. Los receptores actúan en forma alternada. Secuencia normal: ABB, AC, ABB, AC, ABB, AC...

```
SEMAFORO semA = sem(1);
SEMAFORO semB = sem(0);
SEMAFORO semC = sem(0);
SEMAFORO MUTEX = sem(1);

VOID A() {
    BOOL turnoDeB = TRUE;
    While (1) {
        semA.Wait();
        PRODUCIR(); // A PRODUCE DE A DOS items
        IF (turnoDeB) { FORN(i,2) semB.SIGNAL(); }
        ELSE { semC.SIGNAL(); }
        turnoDeB = ! turnoDeB;
    }
}

VOID B() {
    INT COUNT = 0;
    While (1) {
        semB.Wait();
        MUTEX.Wait();
        CONSUMIR DE B();
        MUTEX.SIGNAL();
        COUNT++;
        IF (COUNT == 2)
            { semA.SIGNAL(); }
    }
}

VOID C() {
    While (1) {
        semC.Wait();
        MUTEX.Wait();
        CONSUMIR DE C();
        MUTEX.SIGNAL();
        semA.SIGNAL();
    }
}
```

Ejercicio 9

Suponer que se tienen N procesos P_i , cada uno de los cuales ejecuta un conjunto de sentencias a_i y b_i . ¿Cómo se pueden sincronizar estos procesos de manera tal que los b_i se ejecuten después de que se hayan ejecutado todos los a_i ?

P_1	P_2	...	P_m
a_1	a_2	...	a_m
b_1	b_2	...	b_m

```
int count = 0;
SEMAFORO BARRERA = sem(0);
SEMAFORO MUTEX = sem(1);
void P() {
    a();
    MUTEX.wait();
    count++;
    if (count == m) { for(i, m) { BARRERA.signal(); } }
    MUTEX.signal();
    BARRERA.wait();
    b();
}
```

Ejercicio 10

Se tienen los siguientes dos procesos, `foo` y `bar`, que son ejecutados concurrentemente. Además comparten los semáforos `S` y `R`, ambos inicializados en 1, y una variable global `x`, inicializada en 0.

```
R = sem(1) ; S = sem(1) ;
```

```
int x = 0 ;
```

```
void foo( ) {
```

```
do {
```

```
    semWait(S);
```

```
    semWait(R);
```

```
    x++;
```

```
    semSignal(S);
```

```
    semSignal(R);
```

```
} while (1);
```

```
}
```

```
void bar( ) {
```

```
do {
```

```
    semWait(R);
```

```
    semWait(S);
```

```
    x--;
```

```
    semSignal(S);
```

```
    semSignal(R);
```

```
} while (1);
```

```
}
```

a) ¿Puede alguna ejecución de estos procesos terminar en *deadlock*? En caso afirmativo, describir una traza de ejecución.

Sí, esto potencialmente termina en *deadlock*.

foo	bar	R	S	x
		1	1	0
semWait(S)		1	0	0
	semWait(R)	0	0	0
semWait(R)		0	0	0
	semWait(S)	0	0	0

AMBOS WAITEAN SEMAFOROS QUE NO SERAN 'SEÑALADOS'

b) ¿Puede alguna ejecución de estos procesos generar inanición para alguno de los procesos? En caso afirmativo, describir una traza.

Ejercicio 11 (*Read y Write*)

Se quiere simular la comunicación mediante pipes entre dos procesos mediante las syscalls **read()** y **write()**, pero usando memoria compartida (sin usar **file descriptors**). Se puede pensar al pipe como un buffer de tamaño N, donde en cada posición se le puede escribir un cierto mensaje. El **read()** debe ser bloqueante en caso que no haya ningún mensaje, y si el buffer está lleno, el **write()** también debe ser bloqueante. No puede haber condiciones de carrera y se puede suponer que el buffer tiene los siguientes métodos: **pop()** (saca el mensaje y lo desencola), **push()** (agrega un mensaje al buffer).

Ejercicio 12 (Dividir y conquistar el TP) ★

irge

Un grupo de N estudiantes se dispone a hacer un TP de su materia favorita (Sistemas Operativos).

Cada estudiante conoce a la perfección cómo `implementarTp()` y cómo `experimentar()`. Curiosamente, cada una de estas acciones puede ser llevada a cabo de manera independiente por cada uno, así que decidieron dividirse el trabajo.

Acordaron dividir el trabajo en varias etapas. En cada etapa, todos los estudiantes deben primero `implementarTp()`, y recién cuando todos hayan terminado, pueden empezar a `experimentar()`. Luego, para poder comenzar la siguiente etapa y volver a implementar, todos deben haber terminado de experimentar con la etapa anterior.

Se pide diseñar un programa concurrente que utilice procesos y que modele esta situación utilizando semáforos.

```
INT COUNT = 0;
SEMAFORO MUTEX = SEM(1);
SEMAFORO BARRERA = SEM(0);
SEMAFORO BARRERA2 = SEM(0);
VOID TRABAJO_PRACTICO(M) {
    IMPLEMENTAR TP();
    MUTEX.WAIT();
    COUNT++;
    IF (COUNT == M) { FORM(i, M) { BARRERA.SIGNAL(); }
    MUTEX.SIGNAL();
    BARRERA.WAIT();
    EXPERIMENTAR();
    MUTEX.WAIT();
    COUNT--;
    IF (COUNT == 0) { FORM(i, M) { BARRERA2.SIGNAL(); }
    MUTEX.SIGNAL();
    BARRERA2.WAIT();
}
```

Ejercicio 13 (*Lavarropas automático*) ★

Se tiene un único lavarropas que puede lavar 10 prendas y para aprovechar al máximo el jabón nunca se enciende hasta estar **totalmente lleno**.

Escribir el pseudocódigo que resuelva este problema de sincronización, indicando los semáforos utilizados y sus respectivos valores iniciales, teniendo en cuenta los siguientes requisitos:

- a) El lavarropas invoca `estoyListo()` para indicar que la ropa puede empezar a ser cargada, e invoca `lavar()` una vez que está totalmente lleno. Al terminar el lavado invoca a `puedenDescargarme()`. Una vez vacío, el lavarropas espera prendas nuevamente.
- b) Cada prenda invoca `entroAlLavarropas()` una vez que el lavarropas está listo, y no pueden ingresar dos prendas al lavarropas al mismo tiempo. No es necesario tener en cuenta el orden de llegada de las prendas para introducirlas en el lavarropas, cualquier orden es permitido. Una vez que el lavarropas indicó que puede ser descargado, cada prenda invoca `saquenmeDeAquí()` y termina su proceso. Las prendas pueden salir todas a la vez.